Guia de Estudo de POO – FATEC

Base: repositório **Gabriel-Verdin/poo-fatec** (código das semanas 01 a 10, listas de exercícios, 12 diagramas UML e 3 PDFs de apoio). Todos os caminhos abaixo são relativos à raiz do repositório.

1. Como usar este guia

1. Leia a explicação do tema.
2. Abra o arquivo indicado em **Onde está** e leia o código com a explicação ao lado.
3. Abra o diagrama UML indicado e confira como cada caixa, seta e símbolo virou código.
4. Feche tudo e refaça a classe do zero, sem olhar. Só depois compare.

Aviso honesto: este guia foi montado a partir do que está no repositório. O que realmente cai na prova teórica e na prática é decisão do professor. Os rankings de "mais prováveis" nos PDFs do repositório são estimativas de quem os escreveu, não gabarito.

2. Mapa rápido do repositório

Onde	O que tem	Serve para
`poo/src/semana01`	tipos, casting, operadores, entrada com JOptionPane	base da linguagem
`poo/src/semana02`	for, while, do-while, Math, Random, String	lógica e laços
`poo/src/semana03`	exceções (try/catch), arquivos, janela Swing	tratamento de erros
`poo/src/semana04`	métodos static, retorno, sobrecarga, Javadoc	métodos
`poo/src/semana05`	primeiro objeto (Pincel), toString, recursão	classes e objetos
`poo/src/semana06`	encapsulamento: ContaBancaria, Motor, Pessoa, Televisao	prática e teoria
`poo/src/semana07`	herança, protected, construtores, sobrecarga e sobrescrita, classe abstrata	prática e teoria

Onde	O que tem	Serve para
`poo/src/semana08`	associação, agregação, composição, polimorfismo	prática e teoria
`poo/src/semana09`	interfaces	prática e teoria
`poo/src/semana10`	classe abstrata Motor	teoria
`poo/src/atividadeExercicios`	Listas 1 a 7 resolvidas e a Lâmpada encapsulada	treino
`poo/src/lista01`	os 10 exercícios da Lista 1	treino
`Arquivos/DiagramasUML`	12 diagramas UML	teoria
`Arquivos/Estudo-Metodos.png`	anatomia de um método	teoria
`MateriaisEstudo`	3 PDFs: apoio da prova prática, exercícios prováveis, provas antigas resolvidas	prova prática

3. O que os PDFs dizem sobre a prova prática

- Prova no **Eclipse**, 3 horas. Crie um projeto com o seu nome e, dentro dele, **um package por questão**, com o nome que aparece na aba do diagrama (ex01, venda, caixa...).
- **Toda questão parte de um diagrama UML**; o texto só traz as regras. Quase sempre há uma classe de teste com `main` (App ou UsaXxx).
- Questões de estrutura valem 2,0 a 2,5; as com regras, menu ou banco de dados valem 4,0 a 5,0.
- O professor desconta falta de comentário: `/** */` na classe e em cada método, `//` nas etapas.
- Diagramas costumam omitir construtor e get/set. Pode acrescentar o mínimo necessário, mas **não mude o que está no diagrama**.
- Já caíram: herança, interface com polimorfismo, classe com limite e menu, lista de objetos (associação e agregação), import entre packages e JDBC.
- Atenção: o PDF “Exercícios mais prováveis” diz que herança e polimorfismo ficam fora das listas, mas as duas provas antigas cobram herança, interface e agregação, e o repositório tem as semanas 07 a 10 sobre isso. Estude também.

4. Como ler um diagrama UML (UML para Java)

No diagrama	Significa	Em Java
Retângulo com aba e nome (semana06)	pacote	`package semana06;`
Caixa em 3 partes	nome / atributos / métodos	`class Nome { atributos; métodos }`
`-` antes do nome	privado	`private`
`+` antes do nome	público	`public`
`#` antes do nome	protegido (classe e filhas)	`protected`
`nome : String`	atributo e tipo	`private String nome;`
`soma(a : int) : int`	parâmetro e retorno	`public int soma(int a)`
`: void`	não devolve nada	`public void metodo()`
Nome sublinhado (`main`, `transferir`)	método estático	`public static ...`
Nome em *itálico*	classe ou método abstrato	`abstract`
`<<interface>>`	interface	`public interface X`
Linha cheia, triângulo vazado	herança (“é um”)	`extends`
Linha tracejada, triângulo vazado	realização	`implements`
Tracejada com seta aberta (`<<instance>>`)	dependência: uma classe cria e usa a outra	`new Outra()` dentro do método
Linha com seta aberta	associação navegável (“tem um”)	atributo do tipo da outra classe
Losango **vazado**	agregação	referência recebida de fora
Losango **cheio**	composição	`new` feito dentro da classe
`1`, `0..1`, `1..*`, `4..5`	multiplicidade	`1..*` vira `List<Tipo>`

5. Semana a semana

Semana 01 – Base da linguagem (`poo/src/semana01`)

Ideias

- Tipos primitivos: `byte` , `int` , `float` , `double` , `char` , `boolean` . `String` **não** é primitivo, é objeto (por isso existe `nome.length()`).
- Casting: de tipo menor para maior é automático (byte → int → double). De maior para menor exige `(int)` e corta os decimais: `(int) 123.45` vira 123.
- `final` cria constante. `float` exige o sufixo F (`3.4F`).
- `JOptionPane.showInputDialog` **sempre devolve String**. Converta com `Integer.parseInt` ou `Double.parseDouble` . Cancelar devolve `null` .
- `num++` usa e depois soma; `++num` soma e depois usa. `|` e `&` são operações bit a bit.

Onde está: `Exemplo01` (casting), `Exemplo02` (tipos, final, String), `EntradaDados` (JOptionPane e parse), `Calculadora` e `Tela` (args do main), `Operadores` , `ExemploTernario` (ternário aninhado), `For01` .

Cuidado: em `Operadores.java` os comentários do código dizem 11, 12, 17 e 34, mas o programa imprime 10, 12 e, no fim, **38** (num vai 10, 11, 12, 13, 14, 19, 38). Não decore os comentários.

Semana 02 – Laços, Math e String (`poo/src/semana02`)

Ideias

- `for` : inicialização, condição, **corpo**, incremento (nessa ordem). Use quando sabe quantas voltas.
- `while` : testa antes. `do-while` : executa **pelo menos uma vez**. Use quando não sabe quantas voltas.
- `for` com índice (`n[i]`) e `for-each` (`for (int x : n)`), também para `List<String>` .
- `Math` é estático (`Math.max` , `Math.PI`). `Math.random()` devolve de 0 até menor que 1. Dado de 1 a 6: `1 + (int)(Math.random() * 6)` . `new Random().nextInt(100)` devolve de 0 a 99 (método de objeto, precisa de `new`).
- String: `length` , `charAt` , `substring(ini, fim)` (o fim não entra), `contains` , `toUpperCase` , `split(";")` .

Onde está: `For03` (soma acumulada), `For05` (vetor), `For06` (List), `While01` (`while(true)` com `break`), `While03` (`do-while`), `Math01` a `Math04` , `Senha` e `SenhaAlfanumerica` (sorteio com `charAt`), `St01` (String e split).

Cuidado

- `(int) Math.random()*10` dá sempre 0: o cast vale só para `Math.random()`. O certo é `(int)(Math.random()*10)`.
- `For04` soma `0.1` em `float`: aparecem números como 1.1000001 (imprecisão de ponto flutuante).
- `While02` soma também o último valor digitado (0 ou negativo). Se for negativo, ele **diminui** o total.

Semana 03 – Exceções, arquivos e janela (`poo/src/semana03`)

Ideias

- `try` tenta; `catch` trata o erro; `finally` roda sempre (usado para fechar recursos).
- `NumberFormatException`: texto que não é número. `ArithmeticException`: divisão **inteira** por zero (com `double` dá `Infinity`). `ArrayIndexOutOfBoundsException`: índice fora do vetor.
- Vários `catch`: o mais específico vem primeiro e `Exception` genérica por último. Multi-catch com `|`.
- Padrão de validação que cai muito: `while (true) { try { ... break; } catch (...) { mensagem } }` repete até acertar.
- `throws` passa a responsabilidade para quem chamou. Exceção própria: `class X extends Exception`. `Error` (como falta de memória) não se trata no dia a dia.
- Arquivos: `PrintWriter` grava, `BufferedReader` lê linha a linha.
- Swing: `JFrame` com `JLabel`, `JTextField`, `JButton`; clique tratado com `ActionListener` (classe anônima).

Onde está: `Excecao01` (catch múltiplo e finally), `Excecao02` e `Excecao03` (repetir até válido), `Execao04` (multi-catch), `TrocaPneuException` (exceção própria), `ErroFatal` (estoura a memória), `Arquivo`, `GravaArquivo` e `UsaArquivo` (arquivo), `Frame01` (calculadora gráfica).

Cuidado: `Arquivo.java` e `GravaArquivo.java` usam um caminho absoluto do computador do autor (`/home/gabrielverdin/...`). No seu computador, troque pelo caminho da pasta `Arquivos/alunos.txt`.

Semana 04 – Métodos (`poo/src/semana04`, imagem `Arquivos/Estudo-Metodos.png`)

Anatomia de um método (a imagem desenha isso): `public static void imprimir(String texto)`

- `public`: modificador de visibilidade.

- `static`: pertence à classe; chama-se `Classe.metodo()`. Sem `static`, é preciso criar o objeto (`new`) e chamar `objeto.metodo()`.
- `void`: não devolve nada. Qualquer outro tipo (`int`, `String`) obriga a terminar com `return`.
- `imprimir(...)`: nome + parâmetros formam a **assinatura** (quantidade e tipos dos parâmetros).
- Parâmetros são a porta de entrada; os valores passados na chamada são os argumentos.
- **Sobrecarga:** mesmo nome, assinaturas diferentes, na mesma classe.
- Métodos devem ser documentados com Javadoc (`/** ... @param ... @return ... */`). O README do repositório mostra o comando `javadoc`.

Onde está: `Metodos` (static, sem static, `somarVoid` sobrecarregado, `somarInt` com retorno), `UsaMetodos` (como chamar, e `import semana03.Metodos2` para usar classe de outro package), `Calculadora` (Javadoc completo), `Impressora` (devolve uma String com o resultado), `UsaCalculadora` e `UsaImpressora` (classes de teste), `Cliente` (nome do método sem repetir o nome da classe).

Cuidado: `Calculadora.dividir(int a, int b)` lança `ArithmeticException` se `b` for 0.

Semana 05 – Primeiros objetos (`poo/src/semana05`)

Ideias

- **Classe** é a forma; **objeto** é o que nasce com `new`. Cada objeto tem estado próprio.
- Atributos sem valor inicial nascem com padrão: `null` (String) e `0.0` (double).
- `toString()` é chamado sozinho por `System.out.println(objeto)`. Precisa de `@Override`, porque já existe em `Object`.
- Método de instância (`Impressora.imprimir`) exige objeto; método static não.
- **Recursão:** método que chama a si mesmo e **precisa de condição de parada**, senão estoura a pilha (`StackOverflowError`).

Onde está: `Pincel` e `UsaPincel` (classe e objeto, os comentários `Na UML: + cor: String` mostram a ligação com o diagrama), `Impressora` e `UsaImpressora`, `CalculoMedia` (sobrecarga e vetor como parâmetro), `ExemplosMetodos` (assinaturas variadas), `Recursividade` (somatória e fatorial), `Tabuada`.

Cuidado

- Em `UsaPincel`, o `p2` nunca recebe valores (o código altera o `p1` duas vezes), por isso `p2.mostrar()` imprime `null` e `0.0`.
- `fatorial(0)` e `somatoria(0)` nunca chegam em `n == 1`: dá `StackOverflowError`.

Semana 06 – Encapsulamento (`poo/src/semana06`)

Ideia central: atributos `private` ; quem está fora só mexe neles por métodos públicos; **a regra do enunciado fica dentro do método** (no `set` , no `depositar` ...). Limites viram constantes `final` . É o tema mais provável da prova prática.

ContaBancaria – `ContaBancaria.java` , `TerminalBancario.java` – UML: `UMLContaBancaria.png`

- `titular` é `public` de propósito (exemplo de atributo **não** encapsulado); `numero` e `saldo` são `private` .
- `definirNumero` valida o limite. `depositar` aceita valor maior que 0 e até 10000. `sacar` respeita o limite de 1000 e o saldo.
- `transferir(origem, destino, valor) : String` aparece **sublinhado** no UML porque é `static` (escopo de classe). Os outros são de instância (escopo de objeto).
- Cuidado: se `sacar` falhar por saldo insuficiente, `transferir` mesmo assim chama `depositar` no destino e devolve "sucesso". Na prova, teste se o saque funcionou antes de depositar.

Motor – `Motor.java` , `UsaMotor.java` – UML: `UMLMotor.png`

- `fabricante` público; `velocidade` e `status` privados. `acelerar` só funciona se o motor estiver ligado e abaixo de 100; `frear` não passa de 0.
- A seta tracejada `<<instance>>` de `UsaMotor` para `Motor` é dependência: a classe de teste cria o objeto. O `main` sublinhado é `static` .
- Diferença UML e código: o diagrama mostra `acelerar() : String` , mas o código usa `void` (a versão que devolve String está comentada).

Pessoa – `Pessoa.java` , `UsaPessoa.java` – UML: `UMLPessoa.png`

- 6 atributos privados e 12 métodos get/set (que **não** aparecem no UML, a anotação do próprio diagrama avisa isso) mais `toString()` .

Televisao – `Televisao.java` , `UsaTelevisao.java` – UML: `UMLTelevisao.png`

- Canal de 0 a 999 e brilho só "baixo", "medio" ou "alto", validados no `set` . Compare texto com `equals` ou `equalsIgnoreCase` , **nunca com** `==` .
- O método no código se chama `redizirCanal` (erro de digitação); o UML diz `reduzirCanal` . Na prova, use o nome do diagrama.

Semana 07 – Herança (`poo/src/semana07`)

Ideias

- **Herança** (`extends`): a filha ganha atributos e métodos da mãe e acrescenta os seus. Relação "é um" (Gato é um Animal). Toda classe herda de `Object` (por isso `toString()` e `getClass()` existem).
- **super(...)** chama o construtor da mãe e tem de ser a **primeira linha** do construtor da filha. `super.metodo()` aproveita o que a mãe já faz.
- **Sobrescrita (override):** mesma assinatura na mãe e na filha, com `@Override` (Elefante.comer, Funcionario.mostrar). **Sobrecarga (overload):** mesmo nome, parâmetros diferentes, na mesma classe ou na subclasse (Gato tem `comer()` e `comer(int gramas)`).
- **Construtores:** quando você cria qualquer construtor, o Java **deixa de criar** o construtor vazio. Se ainda quiser, escreva `public Produto() {}` (veja `Produto` e `UsaProduto`).
- **protected:** visível no mesmo package e nas subclasses. Em `Animal` , o `id` é protected: `UsaAnimal` (mesmo package) usa `gato.id` , e `semana06/Cachorro` (outro package) lê `id` por ser filha.
- **Classe abstrata** (`abstract`): não gera objetos, serve de modelo. `new Animal()` não compila.
- Dica de segurança do próprio código: o construtor de `Animal` chama `setIdade(idade)` em vez de `this.idade = idade` , para a validação valer já na criação.

Animal, Gato, Elefante, Cachorro – UML: `UMLAnimal.png`

- Nome em itálico = abstrata. `#` = protected. Setas de Gato, Elefante e Cachorro (esta em outro package, semana06) apontam para Animal.
- Diferenças UML e código: o UML lista `Animal(id, nome, idade)` , mas o código recebe `(id, idade, nome)` . O UML dá `comer()` ao Cachorro; o código tem `mostrar()` . O `setIdade` do código recusa 0 e 120 (usa `>` e `<`), embora as constantes sejam 0 e 120.

Pessoa, PessoaFisica, Funcionario – UML: `UMLPessoaHierarquia.png`

- Cadeia: `Funcionario extends PessoaFisica extends Pessoa extends Object` . Cada nível acrescenta atributo (`cpf` , `salario`) e sobrescreve `digitar()` e `mostrar()` chamando `super` antes de acrescentar o seu.
- `Pessoa` tem 4 construtores sobrecarregados (vazio, `codigo` , `nome` , `codigo` + `nome`) e `toString` sobrescrito.
- `Aluno` , `PessoaJuridica` e `Vendedor` aparecem só no UML, **sem código no repositório**. Ótimo treino: implemente você mesmo (Aluno e Vendedor seguem o mesmo padrão de Funcionario).

MeuBotao e MeuPI – UML: `UMLMeuBotao.png`

- Java **não tem herança múltipla** (uma classe não pode estender JButton e JTextField ao mesmo tempo); o diagrama mostra isso como exemplo do que não existe. A alternativa é usar interfaces (semana 09).
- `MeuBotao extends JButton` muda cor e texto. `MeuPI extends JFrame` usa `ActionListener` anônimo.

Semana 08 – Relacionamentos e polimorfismo (`poo/src/semana08`)

Associação ("tem um", um objeto guarda a referência de outro) – UML: `UMLRelacionamentoAssociacao.png`

- **Reflexiva** (Pessoa com Pessoa): `Pessoa.parceiro` ; veja `UsaPessoa` (o código chama de parceiro, o UML de conjuge).
- **Bidirecional** (Homem e Mulher): `Homem.esposa` e `Mulher.marido` . Em `Casamento.java` os **dois lados** são ligados (`h.esposa = m; m.marido = h;`). Se ligar só um lado, o outro não enxerga.
- **Um para muitos** (Funcionario 1..* e Departamento 1): `Departamento` guarda `List<Funcionario>` com `adicionar(f)` , e `Funcionario` guarda `departamento` . Veja `UsaDepartamento` e `UsaFuncionario` . Multiplicidade `1..*` vira lista.
- **Navegabilidade** (Mesa com Funcionario): a seta mostra que a Mesa conhece o Funcionario, mas não o contrário (`Mesa.f`).
- **Classe associativa** (Casa entre H e M): aparece só no UML, sem código.
- Cuidado: `Funcionario.mostrar()` acessa `departamento.nome` ; se o departamento for `null` dá `NullPointerException` . Use `new ArrayList<>()` na lista, senão `add` falha.

Agregação e composição – `Veiculo` , `Roda` , `Motor` , `Pessoa` , `UsaVeiculo` – UML: `UMLAgregacaoComposicao.drawio.png`

- **Composição** (losango cheio, "posse"): `Roda` e `Motor` são criados **dentro** do Veículo (`public Roda roda = new Roda();`). `new Veiculo()` cria 3 objetos e eles nascem e morrem juntos.
- **Agregação** (losango vazado): a `Pessoa` motorista existe por conta própria e chega **de fora** por `definirMotorista(p)` .
- Diferença UML e código: o UML diz `Roda` 4..5, mas o código guarda só uma `Roda` . Na prova, 4..5 pediria vetor ou `List<Roda>` .

Polimorfismo – `Fruta` , `Banana` , `Abacaxi` , `Melancia` , `Tamara` , `Descascador` – UML: `UMLFruta.png`

- Uma variável da classe mãe guarda um objeto da filha: `Fruta f = new Banana();` `f.descascar();` roda a versão da Banana. Isso só vale porque Banana **é uma** Fruta.
- Vantagem: `Tamara` foi criada depois e o `Descascador` não precisou mudar. O código cliente não fica cheio de `if` por tipo.
- No código `Fruta` é `abstract`; no UML ela aparece sem itálico.

Semana 09 – Interfaces (`poo/src/semana09`)

Ideias

- **Interface** é um contrato: só diz **quais** métodos devem existir, não como funcionam. Seus métodos são sempre públicos e abstratos (sem corpo).
- A classe assina o contrato com `implements` e **é obrigada** a escrever todos os métodos. Pode implementar várias interfaces (`Automovel implements Motor, Capota`), que é como Java contorna a falta de herança múltipla.
- Interface pode estender outra: `Turbo extends Motor`. Quem implementa `Turbo` precisa dos 4 métodos de Motor mais `acionar` e `interromper`.
- Polimorfismo com interface: `Motor m = new Navio();` e depois `m.ligar();`. Quem decide como liga é o objeto.
- Não se cria objeto de interface (`new Motor()` não compila), mas se declara variável dela.

Motor, Navio, Aviao, Automovel, Capota, Ferrari, Turbo, App – UML: `UMLInterfaces.png`

- `<<interface>>` Motor com 4 métodos; Navio, Avião e Automóvel o realizam (seta tracejada, triângulo vazado). Automóvel realiza também Capota (`abrir` , `fechar`): por isso o UML anota “6 métodos obrigatórios”.
- `Ferrari` herda de `Veiculo` (seta cheia, vem de `semana08`) **e** realiza `Turbo`. No código: `class Ferrari extends Veiculo implements Turbo`. A ordem é sempre `extends` primeiro, `implements` depois.
- `Carro` (filha de Automóvel no UML) não tem código no repositório.

Interface fornecida e requerida – `Fisio` , `Pessoa` , `Animal` , `ET` , `Vida` , `UsaFisio` – UML: `UMLInterfaceProvedora.png`

- O “pirulito” (círculo) é a interface **fornecida** pelas classes `ET` , `Pessoa` e `Animal`. A “meia-lua” é a interface **requerida** por `Vida`.
- No código: `Vida.viver(Fisio f)` só chama `comer()` e `beber()`; ela funciona com qualquer coisa que implemente `Fisio`. `UsaFisio` testa com os três.
- `Pessoa` e `Animal` dessa semana são outras classes, diferentes das de `semana07` e `semana08`.

PI.java: uma janela que implementa `ActionListener`, `MouseListener` e `KeyListener`. Obriga a escrever os 9 métodos dessas interfaces, mesmo vazios.

Semana 10 – Classe abstrata (`poo/src/semana10`)

- `Motor` é `abstract`: tem atributos (`potencia`, `modelo`), um método concreto (`mostrar`) e quatro `abstract` (`ligar`, `desligar`, `acelerar`, `frear`). `Moto extends Motor` precisa implementar os quatro.
- Não há diagrama desta semana. Não confunda com `UMLMotor.png` (semana 06) nem com a interface `Motor` (semana 09): são três classes diferentes com o mesmo nome.

	Classe abstrata	Interface
Palavra-chave	`extends` (só uma)	`implements` (várias)
Atributos	pode ter	só constantes
Métodos	abstratos e concretos	abstratos (sem corpo)
Construtor	pode ter	não tem
Quando usar	filhas compartilham código e estado (“é um”)	classes sem parentesco com a mesma capacidade (“pode fazer”)

6. Tabela mestre: diagrama UML → código

Todos os diagramas estão em `Arquivos/DiagramasUML/`.

Diagrama	O que ensina	Onde está o código	Diferenças para anotar
`UMLPessoa.png`	classe simples; get/set não aparecem	`semana06/Pessoa.java`	–
`UMLContaBancaria.png`	encapsulamento, método `static` sublinhado	`semana06/ContaBancaria.java`, `TerminalBancario.java`	–
`UMLMotor.png`	dependência `<<instance>>`, `main` estático	`semana06/Motor.java`, `UsaMotor.java`	UML: `acelerar() : String`; código: `void`
`UMLTelevisao.png`	encapsulamento com limites	`semana06/Televisao.java`, `UsaTelevisao.java`	UML `reduzirCanal`; código `redizirCanal`
`UMLAnimal.png`	herança, `#protected`, classe abstrata (itálico), sobrecarga, outro package	`semana07/Animal.java`, `Gato.java`, `Elefante.java`, `semana06/Cachorro.java`	ordem do construtor; `Cachorro.comer()` não existe no código

Diagrama	O que ensina	Onde está o código	Diferenças para anotar
`UMLPessoaHierarquia.png`	hierarquia de 3 níveis, `super`, sobrescrita, `Object`	`semana07/Pessoa.java`, `PessoaFisica.java`, `Funcionario.java`	`Aluno`, `PessoaJuridica`, `Vendedor` sem código
`UMLMeuBotao.png`	herança múltipla não existe em Java	`semana07/MeuBotao.java`, `MeuPI.java`	é um contra-exemplo; o código usa herança simples
`UMLRelacionamentoAssociacao.png`	associação: navegabilidade, 1..*, reflexiva, bidirecional, classe associativa	`semana08/Mesa.java`, `Funcionario.java`, `Departamento.java`, `Pessoa.java`, `Homem.java`, `Mulher.java`, `Casamento.java`	atributo `conjuge` no UML, `parceiro`, `esposa`, `marido` no código; classe `Casa` sem código
`UMLAgregacaoComposicao.drawio.png`	losango cheio (composição) e vazado (agregação)	`semana08/Veiculo.java`, `Roda.java`, `Motor.java`, `Pessoa.java`, `UsaVeiculo.java`	UML: `Roda` 4..5; código: uma `Roda`
`UMLFruta.png`	herança e polimorfismo	`semana08/Fruta.java`, `Banana.java`, `Abacaxi.java`, `Melancia.java`, `Tamara.java`, `Descascador.java`	`Fruta` é `abstract` no código
`UMLInterfaces.png`	interface, `implements`, interface que estende interface, várias interfaces	`semana09/Motor.java`, `Turbo.java`, `Capota.java`, `Navio.java`, `Aviao.java`, `Automovel.java`, `Ferrari.java`, `App.java`	`Carro` sem código
`UMLInterfaceProvedora.png`	interface fornecida e requerida (pirulito e meia-lua)	`semana09/Fisio.java`, `Pessoa.java`, `Animal.java`, `ET.java`, `Vida.java`, `UsaFisio.java`	os campos e métodos do diagrama são genéricos ("field: type")
`Estudo-Metodos.png`	anatomia do método, `void` e `return`, sobrecarga	`semana04/Metodos.java`, `UsaMetodos.java`, `Calculadora.java`	–

7. Provas antigas e onde treinar cada padrão

As duas provas antigas estão resolvidas em `MateriaisEstudo/POO_Provas_Anteriores_Resolvidas.pdf` (diagrama, interpretação, código e armadilhas). Esta tabela liga cada questão ao que você já tem no código do repositório.

Questão	Padrão cobrado	Treine no repositório
Prova 1, Q1 (2,0): `Veiculo` e `Ferrari`	herança, `super(...)`, `@Override` com `super.mostrar()`, validação 0 a 350	`semana07`: Pessoa → PessoaFisica → Funcionario

Questão	Padrão cobrado	Treine no repositório
Prova 1, Q2 (4,0): `Pedido` e `FormaPagamento`	interface, `Pix` e `Credito` com `implements`, polimorfismo, JOptionPane	`semana09`: `Vida.viver(Fisio)` e `App` com `Motor m = new Navio()`
Prova 1, Q3 (4,0): `CaixaDeAgua` e `Bomba`	constante, limite de capacidade, métodos que devolvem `int`, menu `do-while` com `switch`	`semana06`: `ContaBancaria` e `Motor` (limites); `atividadeExercicios/Lista06Exercicio05` (menu de operações)
Prova 2, Q1 (2,5): `Proprietario` com `List<Veiculo>`	associação 1 para 1..*, lista, `for-each`	`semana08`: `Departamento` e `UsaDepartamento`
Prova 2, Q2 (2,5): `Frota`	agregação, `import ex01.Veiculo;` (classe de outro package), `do-while` "deseja continuar?"	`semana08/Departamento`; import de outro package em `semana04/UsaMetodos`
Prova 2, Q3 (5,0): banco de dados com JDBC	`DriverManager`, `PreparedStatement` com `?`, `executeUpdate` e `executeQuery`, `while (rs.next())`	**não há código de JDBC no repositório**; estude as páginas 31 a 40 do PDF

JDBC em uma frase por etapa: conectar (`DriverManager.getConnection`), preparar (`prepareStatement` com `?`), preencher (`setInt`, `setString`...), executar (`executeUpdate` para INSERT, `executeQuery` para SELECT), ler (`while (rs.next())` com `getX`), fechar (`try` com parênteses). O `.jar` do driver precisa estar no Build Path.

8. Esqueleto de classe encapsulada (o que mais cai na prática)

O PDF de apoio manda decorar esta ordem: constantes, atributos, construtor, get/set, comportamento, `toString`. O construtor chama os `set`, para a validação valer já na criação.

```
package apoio;

/** Modelo de aluno com validação. */
public class Aluno {
    public static final double NOTA_MIN = 0;
    public static final double NOTA_MAX = 10;

    private String nome;
    private double nota;

    public Aluno(String nome, double nota) {
        setNome(nome);
        setNota(nota);
    }

    public String getNome() { return nome; }
    public void setNome(String nome) {
        if (nome != null && !nome.trim().isEmpty()) {
            this.nome = nome;
        } else {
            System.out.println("Nome inválido.");
        }
    }

    public double getNota() { return nota; }
    public void setNota(double nota) {
        if (nota >= NOTA_MIN && nota <= NOTA_MAX) {
            this.nota = nota;
        } else {
            System.out.println("Nota inválida: " + nota);
        }
    }

    public String situacao() {
        return nota >= 7 ? "Aprovado" : "Reprovado";
    }

    @Override
    public String toString() {
        return "Nome: " + nome + " | Nota: " + nota + " | " + situacao();
    }
}
```

Classe de uso: lê com `JOptionPane.showInputDialog`, converte com `Integer.parseInt` ou `Double.parseDouble` (decimal com **ponto**), cria o objeto e mostra com `JOptionPane.showMessageDialog(null, objeto)` e `System.out.println(objeto)`. Versão completa (com `UsaAluno` e `Validacao`) em `MateriaisEstudo/Material de apoio prova prática de POO.pdf`.

No repositório, a Lâmpada encapsulada (`atividadeExercicios/EncapLampada.java`) guarda o estado em um `enum` (`EncapEstadoLampada`) em vez de constantes String. Funciona, mas se o enunciado pedir constante String com `set` validando, adapte o esqueleto acima.

9. Perguntas de teoria com resposta curta

1. **Classe e objeto?** Classe é o molde (atributos e métodos); objeto é uma instância criada com `new`, com estado próprio.
2. **O que é encapsulamento e para que serve?** Esconder os atributos (`private`) e liberar o acesso por métodos (get/set), que validam os valores. Protege o estado do objeto.
3. **Construtor?** Método com o nome da classe e sem tipo de retorno, chamado no `new`. Se você não escrever nenhum, o Java cria um vazio; se escrever qualquer um, o vazio deixa de existir.
4. **`this` e `super`?** `this` é o próprio objeto (`this.nome = nome`). `super` é a classe mãe (`super(...)` chama o construtor dela, na primeira linha; `super.metodo()` usa a versão da mãe).
5. **Sobrecarga e sobrescrita?** Sobrecarga: mesmo nome com parâmetros diferentes. Sobrescrita: mesma assinatura na filha, trocando o comportamento da mãe (`@Override`).
6. **Herança?** A filha reaproveita atributos e métodos da mãe (`extends`). Java só permite **uma** classe mãe.
7. **Modificadores?** `public` (todos), `private` (só a classe), `protected` (mesmo package e subclasses), sem modificador (só o mesmo package).
8. **`static`?** Pertence à classe, não ao objeto: `Math.pow(2, 3)`. Método sem `static` precisa de objeto.
9. **Classe abstrata e interface?** Veja a tabela da semana 10. Nenhuma das duas gera objeto com `new`.
10. **Polimorfismo?** Uma variável da mãe ou da interface guarda objetos de filhas ou implementações, e a chamada roda a versão do objeto real (`Fruta f = new Banana()`).

11. **Associação, agregação e composição?** Associação: um objeto conhece outro. Agregação: o todo tem partes que existem sozinhas (motorista). Composição: as partes nascem e morrem com o todo (roda e motor criados dentro do veículo).
12. **Multiplicidade `1..*`?** Um ou mais; implemente com `List<Tipo>` e `new ArrayList<>()`.
13. **`==` e `equals`?** `==` compara referência; texto se compara com `equals` ou `equalsIgnoreCase`.
14. **Para que serve o `toString`?** Define o texto do objeto em `println`. Vem de `Object`, por isso usa `@Override`.
15. **`try`, `catch` e `finally`?** `try` tenta, `catch` trata um tipo de erro, `finally` roda sempre. `throws` passa o erro adiante.
16. **Por que o construtor deve chamar o `set`?** Para a mesma validação valer na criação do objeto (veja o construtor de `Animal`).

10. Armadilhas e erros que existem no próprio repositório

Na hora de estudar, não copie estes pontos como se estivessem corretos:

- `atividadeExercicios/Lista05Exercicio10`: `while (frase.contains("as")) { contador++; }` nunca altera `frase`, então é um **laço infinito**.
- `atividadeExercicios/Lista03Exercicio02`: a média usa divisão inteira (`soma / count`), perde os decimais e dá `ArithmeticException` se nenhum número válido for digitado.
- `atividadeExercicios/Lista02Exercicio09`: a faixa `anos > 75` deixa o valor 75 cair no `else`, que calcula “faltam -50 anos”.
- `semana06/ContaBancaria.transferir`: deposita no destino mesmo quando o saque falha.
- `semana07/Animal.setIdade`: recusa 0 e 120 apesar de as constantes serem 0 e 120.
- `semana02/While02`: soma o último valor (zero ou negativo).
- `semana05/UsaPincel`: `p2` nunca recebe valores.
- `semana01/Operadores`: comentários com resultados errados (o certo termina em 38).
- `semana03/Arquivo` e `GravaArquivo`: caminho absoluto do computador do autor.
- `semana05/Recursividade`: sem tratar 0, `fatorial(0)` estoura a pilha.
- Nomes: `redizirCanal` (UML: `reduzirCanal`), `Execao04`, `Excecao`... Na prova, siga sempre o nome do diagrama.

11. Plano de estudo sugerido

1. **Base (semanas 01 a 05):** leia o guia, rode `Metodos`, `UsaMetodos`, `UsaPincel` e `CalculoMedia`. Refaça `Excecao02` (pedir de novo até ser válido) sem olhar.
2. **Encapsulamento (semana 06):** refaça `Televisao`, `Motor` e `ContaBancaria` do zero olhando só o UML. Depois faça a Lampada, `Produto` e `Tv` do PDF "Exercícios mais prováveis".
3. **Herança (semana 07):** refaça `Animal`, `Gato`, `Elefante`; depois implemente `Aluno` e `Vendedor` do `UMLPessoaHierarquia.png`, que não têm código.
4. **Relacionamentos (semana 08):** refaça `Departamento` e `Funcionario` (lista), `Veiculo` (composição e agregação) e `Descascador` (polimorfismo).
5. **Interfaces (semanas 09 e 10):** refaça `Motor` (interface) com `Navio` e `Aviao`, depois `Fisio` e `Vida`. Compare com `Motor` abstrato da semana 10.
6. **Provas antigas:** resolva as 6 questões do PDF **sem olhar a solução**, cronometrando. Comece pelas de 4,0 e 5,0 pontos.
7. **Teoria:** responda em voz alta a lista da seção 9 e desenhe de memória os diagramas das semanas 07 a 09.

Checklist do dia da prova prática (resumo do PDF): projeto com seu nome; um package por questão com o nome do diagrama; cada atributo com o modificador e o tipo do diagrama; cada método com nome, parâmetros e retorno do diagrama (e `return` quando não for `void`); `extends`, `implements` ou lista conforme a seta; regras do enunciado dentro dos métodos; classe de teste com `main`; testar um valor válido e um inválido; comentários `/** */`; nenhum X vermelho no Package Explorer.